Lab No. 02

The Linux Shell

Instructions:

- Read the lab carefully, and follow through with exercises
- Answer the concept checks highlighted in yellow and questions highlighted in green.
- Solve the graded tasks, and submit the screenshots of the commands executed.

Submission Guidelines:

- Make a single pdf or docx document and answer all the concept checks ,questions, and graded tasks.
- Use the following naming convention: *bscs23xxx_lab02*

What is the shell?

Computers these days have a variety of interfaces for giving them commands; fanciful graphical user interfaces, voice interfaces, and even AR/VR are everywhere. These are great for 80% of use-cases, but they are often fundamentally restricted in what they allow you to do — you cannot press a button that isn't there or give a voice command that hasn't been programmed. To take full advantage of the tools your computer provides, we have to go old-school and drop down to a textual interface: The Shell.

Nearly all platforms you can get your hands on have a shell in one form or another, and many of them have several shells for you to choose from. While they may vary in the details, at their core they are all roughly the same: they allow you to run programs, give them input, and inspect their output in a semi-structured way.

In this lab, we will focus on the Bourne Again SHell, or "bash" for short. This is one of the most widely used shells, and its syntax is similar to what you will see in many other shells. To open a shell *prompt* (where you can type commands), you first need a *terminal*. Your device probably shipped with one installed, or you can install one fairly easily.

Using the Shell:

The Linux shell (commonly Bash) is a command-line interface that allows users to interact directly with the operating system. Every command entered into the shell follows a general pattern: **command [options] [arguments]**. The *command* specifies the action to perform, *options* (usually preceded by a dash `-` or double dash `--`) modify the behavior of the command, and *arguments* specify the targets on which the command operates, such as files, directories, or processes. For example, in `ls -l /home`, `ls` is the command, `-l` is the option (long listing format), and `/home` is the argument (the directory to list)

When you launch your terminal, you will see a *prompt* that often looks a little like this:

```
missing:~$
```

This is the main textual interface to the shell. It tells you that you are on the machine **`missing`** and that your "current working directory", or where you currently are, is **`~` (short for "home")**. The **`$`** tells you that you are not the root user (more on that later). At this prompt you can type a *command*, which will then be interpreted by the shell. The most basic command is to execute a program:

```
missing:~$ date
Fri 10 Jan 2020 11:49:31 AM EST
missing:~$
```

Here, we executed the `date` program, which (perhaps unsurprisingly) prints the current date and time. The shell then asks us for another command to execute. We can also execute a command with *arguments*:

```
missing:~$ echo hello
hello
```

In this case, we told the shell to execute the program `echo` with the argument `hello`. The `echo` program simply prints out its arguments. The shell parses the command by splitting it by whitespace, and then runs the program indicated by the first word, supplying each subsequent word as an argument that the program can access. If you want to provide an argument that contains spaces or other special characters (e.g., a directory named "My Photos"), you can either quote the argument with `'` or `"` (`"My Photos"`), or escape just the relevant characters with `\` (`My\ Photos`).

But how does the shell know how to find the `date` or `echo` programs? Well, the shell is a programming environment, just like Python or Ruby, and so it has variables, conditionals, loops, and functions. When you run commands in your shell, you are really writing a small bit of code that your shell interprets. If the shell is asked to execute a command that doesn't match one of its programming keywords, it consults an *environment variable* called `$PATH` that lists which directories the shell should search for programs when it is given a command:

```
missing:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
missing:~$ which echo
/bin/echo
missing:~$ /bin/echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

When we run the `echo` command, the shell sees that it should execute the program `echo`, and then searches through the `:`-separated list of directories in `$PATH` for a file by that name. When it finds it, it runs it (assuming the file is *executable*; more on that later). We can find out which file

is executed for a given program name using the `which` program. We can also bypass `$PATH` entirely by giving the *path* to the file we want to execute.

If you don't know a command, no need to worry—Linux provides several ways to get help directly from the shell. The most common is the **`man` (manual)** command, which shows the detailed documentation for a command, e.g., `man ls`. For a quick summary of available options, many commands also support **`--help`**, e.g., `ls --help`. You can use **`whatis`** to get a one-line description of a command, or **`apropos`** to search the manual pages for keywords if you don't remember the exact name of a command. With these tools, you can easily explore commands and learn how to use them without leaving the terminal.

For this Lab, we will work on the following categories of commands:
1. **File & Directory Navigation**
2. **File & Directory Management**
3. **File Viewing & Content Inspection**
4. **File Permissions & Ownership**
5. **Redirection & Piping**

File & Directory Navigation:

Learning Goals:

- **Understand the structure of the Linux filesystem (`/`, `/home`, current directory, parent directory).**
- **Distinguish between absolute and relative paths.**
- **Use commands to move around, inspect, and orient themselves.**

Commands Covered:

1. **`pwd` –** Print working directory

2. `ls` – List files and directories
3. `cd` – Change directory

Introduction:
A path on the shell is a delimited list of directories; separated by `/` on Linux and macOS and `\` on Windows. On Linux and macOS, the path `/` is the “root” of the file system, under which all directories and files lie, whereas on Windows there is one root for each disk partition (e.g., `C:\`). We will generally assume that you are using a Linux filesystem in this class. A path that starts with `/` is called an *absolute* path. Any other path is a *relative* path. Relative paths are relative to the current working directory, which we can see with the `pwd` command and change with the `cd` command. In a path, `.` refers to the current directory, and `..` to its parent directory**:**

```
missing:~$ pwd
/home/missing
missing:~$ cd /home
missing:/home$ pwd
/home
missing:/home$ cd ..
missing:/$ pwd
/
missing:/$ cd ./home
missing:/home$ pwd
/home
missing:/home$ cd missing
missing:~$ pwd
/home/missing
missing:~$ ../../bin/echo hello
hello
```

Notice that our shell prompt kept us informed about what our current working directory was. In general, when we run a program, it will operate in the current directory unless we tell it otherwise. For example, it will usually search for files there, and create new files there if it needs to.

To see what lives in a given directory, we use the `ls` command:

```
missing:~$ ls
missing:~$ cd ..
missing:/home$ ls
missing
missing:/home$ cd ..
missing:/$ ls
bin
boot
dev
etc
home
...
```

Exercises:

Step 1: Orientation with pwd and ls:

1. Run `pwd` to see where you are in the filesystem.
2. Use `ls` to list files. Compare `ls`, `ls -l`, and `ls -lh` and **answer the following questions**
    a. **Which columns appear in long format?**
    b. **What changes with `-h`?**
3. Run `ls -a` and notice hidden files (those starting with `.`).
4. From anywhere, run `ls /` to view the root directory contents.

Concept Check: Which of these is an absolute path: `/home/student`, `../Downloads`?

Step 2: Moving Around with `cd`

5. Use cd / to move to the root directory, then confirm with pwd
6. Return to your home directory using:
    - cd

- cd ~
- cd /home/<your-username>
  Compare — which are absolute vs. relative?

7. Inside your home directory, use `cd Documents` and then run `pwd`. Now try:
   - `cd ../Downloads` (relative path)
   - `cd ~/Downloads` (absolute path using `~`)

***Concept check*: What do `..` and `.` mean in paths?**

Step 3: Practice with nested directories

8. Use the following command to create a nested directory structure:
   mkdir -p ~/Lab/nav/level1/level2/level3 (more on mkdir later)
9. Navigate directly to `level3` using an **absolute path**.
10. From home, navigate to the same directory using **relative paths only**.
11. From `level3`, move back up two levels in a single command (`cd ../..`)

File and Directory Management:

Learning Objectives:

1. Learn how to create, copy, move/rename, and delete files and directories.
2. Understand the difference between files and directories.
3. Safely manage files using commonly used options.

Introduction:

Once you know how to navigate the filesystem, the next step is learning how to manage files and directories. In Linux, everything is treated as a file, whether it's a regular document, a directory, or even a device. The shell provides simple commands to create, copy, move, rename, and delete files and directories. By mastering these operations, you gain full control over organizing and maintaining your workspace, which is essential for any further work in the shell.

Commands Covered:

1. touch - Create empty files or update timestamps
2. mkdir - Create directories (`-p` for parent dirs)
3. cp - Copy files or directories (`-r` for recursive)
4. mv - Move or rename files/directories

5. rm - Remove files or directories (`-r` for recursive, `-i` for interactive)

Step 1: Creating Files and Directories

1. In your home directory, create a directory `Lab/files`.

   mkdir -p ~/Lab/files

2. Move into it(using cd) and create three files:

   touch file1.txt file2.txt file3.txt

3. Verify with `ls -l`. What information does it show?

***Concept check*: What does `touch` do if the file already exists?**

Step 2: Copying Files and Directories:

4. Copy `file1.txt` to a new file called `file1_copy.txt`.

   cp file1.txt file1_copy.txt

5. Create a new directory `backup` inside `Lab/files` and copy all `.txt` files into it

   mkdir backup

   cp *.txt backup/

Check in: What is the role of * in the above command?

6. Try copying an entire directory:

   cp -r backup backup_copy

***Concept check*: Why do we need `-r` when copying directories?**

Step 3: Moving and Renaming:

7. Rename `file2.txt` → `notes.txt`.

   mv file2.txt notes.txt

8. Move `notes.txt` into the `backup` directory.

   mv notes.txt backup/

9. Rename the `backup_copy` directory to `archive`.

***Concept check*: How does `mv` act as both move and rename?**

Step 4: Removing Files and Directories

10. Delete `file3.txt` with:

    rm file3.txt

11. Try deleting a non-empty directory (`archive`). What happens?
12. Use `rm -r archive` to remove it recursively.

Concept check: Why is `rm -rf` considered dangerous?

File Viewing & Content Inspection

Once files are created, the next skill is learning how to view and inspect their contents. The shell offers multiple commands for reading files in different ways—whether you want to quickly check a few lines, scroll through large files, or examine detailed information like size, type, and word count. These tools help you efficiently explore and analyze files without needing a text editor.

Commands Covered:

- `cat` → **Display entire file contents**
- `less`, `more` → **Scroll through file (paged viewing)**
- `head` → **Show beginning of file**
- `tail` → **Show end of file (`tail -f` for live updates)**
- `wc` → **Count words, lines, characters, bytes**
- `file` → **Identify file type**
- `stat` → **Show detailed file information (permissions, size, timestamps)**

Step 1: Displaying Files

1. Create a sample file with some text. Use the following commands:

   echo "Hello World" > sample.txt
   echo "This is line 2" >> sample.txt
   echo "This is line 3" >> sample.txt

   Ignore the “>” and “>>” operators for now, more on them later!

2. Use `cat sample.txt` to display the contents.
3. Try `cat -n sample.txt` to number each line.

Step 2: Viewing Large Files

4. Use `less sample.txt` and practice:
   Spacebar (scroll down)
   `b` (scroll up)
   `/word` (search for “word”)
   `q` (quit)
5. Compare with `more sample.txt`.

Step 3: Checking File Start and End:

6. Use `head -n 2 sample.txt` to show the first 2 lines.
7. Use `tail -n 2 sample.txt` to show the last 2 lines.
8. Run `tail -f /var/log/syslog` (or another log file) to watch updates live.

File Permissions & Ownership:

In Linux, every file and directory has an associated set of **permissions** and **ownership** that control who can access or modify it. Each file belongs to a **user (owner)** and a **group**, and permissions are divided into three categories: **read (r), write (w), and execute (x)** for the

owner, group, and others. Permissions can be viewed with `ls -l` and modified using `chmod`. Ownership can be changed with `chown` (change owner) and `chgrp` (change group). Understanding permissions is critical for system security and proper collaboration.

Commands Covered:
`ls -l` → View permissions and ownership
`chmod` → Change file permissions (symbolic `u/g/o` or numeric `755`)
`chown` → Change file owner
`chgrp` → Change group ownership
`umask` → Show/change default permissions for new files

MUST READ:
In Linux, every file and directory has a set of permissions and ownership that can be viewed with `ls -l`. A typical line of output looks like `-rwxr-xr-- 1 user group size date filename`. The first character indicates the type: `-` for a regular file, `d` for a directory, `l` for a symbolic link, and so on. The next nine characters are grouped into three sets of three (`rwx`), representing the permissions for the **owner**, the **group**, and **others**. Here, `r` means read, `w` means write, and `x` means execute (or "search" in the case of directories). A dash (`-`) indicates absence of that permission. For files, `r` allows viewing contents, `w` allows modifying them, and `x` allows executing them as programs. For directories, `r` allows listing contents, `w` allows creating or deleting entries, and `x` allows entering (traversing) the directory. For example, in a directory with permissions `drwxr-x---`, the owner has full control, members of the owning group can read and enter it, and everyone else has no access. This system ensures fine-grained control over who can read, modify, or execute files and directories.

1. Run `ls -l` on your home directory and interpret the permission string (`-rw-r--r--`).
2. Create a new file, **notes.txt**.
3. Remove write permission for others and verify with ls `ls -l`
   chmod o-w notes.txt
4. Add execute permission for the owner:
   chmod u+x notes.txt
5. Change permissions numerically:
   chmod 644 notes.txt # rw-r--r--
   chmod 755 notes.txt # rwxr-xr-x

Check in:

1. ***What's the difference between symbolic and numeric (`chmod 755`) permission changes?***
2. ***If a file has `----------` (no permissions), can the owner still delete it? Why or why not?***

Redirection & Piping:

In Linux, programs communicate with their environment through *streams*—specifically, **standard input (stdin)**, **standard output (stdout)**, and **standard error (stderr)**. By default, stdin comes from the keyboard, and stdout/stderr are displayed on the terminal. However, the shell allows us to **redirect** these streams to files or other commands, making Linux extremely flexible for handling data. In addition, **pipes (`|`)** let us connect multiple commands so the output of one becomes the input of another, forming powerful command pipelines.

Redirection:

Redirection allows you to control where input and output go. By default, commands read from the keyboard (stdin) and display output/errors on the terminal (stdout/stderr). Using `>` you can save command output into a file, e.g., `ls -l > file_list.txt` stores the long listing of files for later viewing. `>>` appends instead of overwriting, which is useful if you're keeping logs of permission changes, e.g., `chmod 644 notes.txt >> changes.log`. Input can also be redirected: `wc -l < file_list.txt` counts lines from the saved listing instead of typing it again. This makes it easy to capture results and reuse them.

```
missing:~$ echo hello > hello.txt
missing:~$ cat hello.txt
hello
missing:~$ cat < hello.txt
hello
missing:~$ cat < hello.txt > hello2.txt
missing:~$ cat hello2.txt
hello
```

Piping:

Piping (`|`) links commands so the output of one becomes the input of another, letting you combine tools. For example, `ls -l | grep rwx` shows only files where the permission string contains `rwx`. Similarly, `cat /etc/passwd | less` lets you view system user information one screen at a time. You can also filter ownership information: `ls -l | grep username` displays files owned by a specific user. Pipes encourage you to build quick, powerful chains instead of running multiple separate commands.

Tasks:

1. Run `ls -l > list.txt` and then open `list.txt` with `cat` to verify the output was saved.
2. Use `echo "This is a test line" >> list.txt` to append a new line at the end of the same file.
3. Redirect input to `wc -l` by running `wc -l < list.txt` and confirm it shows the number of lines in your saved listing.
4. Run a command that produces an error (e.g., `ls nofile 2> errors.txt`) and check that the error message was saved in `errors.txt`.

Note that for task 4, we use 2> instead of just >. This is because the stderr, i.e the standard error stream is numbered 2 in Linux. Essentially what we are saying in that command is "Whatever the error of this command is, don't display it on the terminal, rather write the error in a file named errors.txt"

Piping Tasks:

1. Run `ls -l | less` to view the long listing one screen at a time.
2. Use `ls -l | grep ".txt"` to display only `.txt` files from the directory.
3. Combine multiple pipes: `ls -l | grep ".txt" | wc -l` to count how many `.txt` files exist.
4. Display the first 5 lines of `/etc/passwd` neatly by running `cat /etc/passwd | head -n 5`.

Putting it all together:

Now that you've learned the basics of navigating the filesystem, managing files, inspecting content, working with permissions, and using redirection and piping, it's time to combine these skills. Real-world Linux usage often involves chaining commands to perform more complex tasks efficiently. In this section, we'll explore a few examples of such combined commands and break down what each part does, so you can see how everything fits together.

1. Find all `.c` files in `/usr/include` and count them
   ls -R /usr/include | grep "\.c$" | wc -l

   What Happens:
   - `ls -R /usr/include` → recursively lists all files in `/usr/include` and its subdirectories.
   - `grep "\.c$"` → filters only filenames ending with `.c`.
   - `wc -l` → counts how many `.c` files there are.
2. Display top 5 largest files in `/var/log`
   `ls -lhS /var/log | head -n 5`

   What happens:
   - `ls -lhS /var/log` → lists files in human-readable format, sorted by size (largest first).
   - `head -n 5` → shows only the top 5 results

Concept Check:

1. **Explore grep and list down what it might be useful for.**
2. **Try using the && and || operators between commands. For instance: cd ~/Downloads && mkdir New.**
3. **All commands return an exit code after successful or unsuccessful execution. What is the exit code for a successful execution and what are the exit codes for unsuccessful execution? How can we check the status codes?**

GRADED TASKS:

Kindly note you are likely to encounter command(s) not covered in this lab. Don't panic, just use

man <command> or *<command> - -help* to get the details of the said command.

1. **View only the first 5 lines of every `.log` file in `/var/log`**
    - You'll need to list the files, and then redirect or pipe the output into a command that prints the first 5 lines.. *Hint:* explore **grep**
2. **Find and count `.txt` files in your home directory**
    - Use a command that lists all `.txt` files in your home directory (and its subdirectories) and then count how many there are. *Hint:* Combine `find` with `wc -l`.
3. **Search for the word "error" in all `.log` files and save results to a new file**
    - The command should look inside every `.log` file in `/var/log`, find lines containing "error", and write them into a file `errors_found.txt` in your home directory.
4. **Find the 5 largest files in your home directory**
    - Use a command that lists files with their sizes, sorts them in descending order, and shows the top 5.
    - *Hint:* Think about `du`, `sort`, and `head`**.**

Bonus Task:

Create a single command that will:

1. **Navigate into your home directory.**
2. **List only the directories (not files) that you own (have permission for).**
3. **For each directory, count how many files it contains (not recursive).**
4. **Save this output to a file called `dir_summary.txt`.**
5. **Then display the contents of `dir_summary.txt` to the screen, sorted in descending order of file count.**

Hints:

- **Use `ls -l` with filtering for directories**
- **Use `wc -l` for counting**
- **Redirection (`>` and `>>`) to build the output file.**
- **Piping + `sort -nr` for ordering.**